EXP 3	**Implement a Generative Adversarial Network (GAN) to generate synthetic images or tabular data. Dataset: MNIST (for images) or Synthetic Financial Transactions.**

Generative AI

AIM:

To implement a **Generative Adversarial Network (GAN)** to generate **synthetic images** using the **MNIST dataset**, where the model learns to create realistic handwritten digits.

Algorithm :

- Load the MNIST dataset
- Normalize the image data
- Create the Generator model
- Create the Discriminator model
- Combine both models to form GAN
- For each training epoch:
- Generate fake images using Generator
- Take real images from dataset
- Train Discriminator on real and fake images
- Train Generator to fool Discriminator
- Repeat training for many epochs
- Use Generator to produce new synthetic images

Code & Output :

**EXP - 3 **

Implement a Generative Adversarial Network (GAN) to generate synthetic images or tabular data. Dataset: MNIST (for images) or Synthetic Financial Transactions.

```
!pip install torch torchvision matplotlib

import torch
import torch.nn as nn
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.25.0+cpu)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.25.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
```

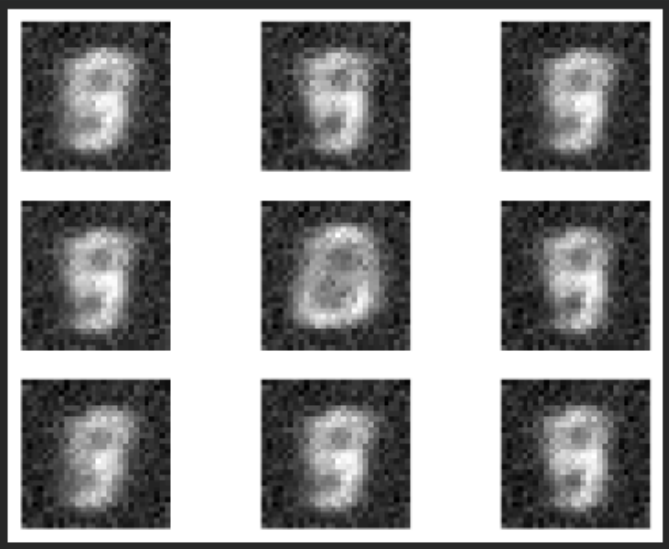
```
data = datasets.MNIST('.', download=True,
                      transform=transforms.ToTensor())

loader = torch.utils.data.DataLoader(data, batch_size=64, shuffle=True)
```

```python
G = nn.Sequential(
    nn.Linear(100, 128),
    nn.ReLU(),
    nn.Linear(128, 784),
    nn.Tanh()
)

D = nn.Sequential(
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Linear(128, 1),
    nn.Sigmoid()
)
```

```python
loss = nn.BCELoss()
g_opt = torch.optim.Adam(G.parameters(), 0.001)
d_opt = torch.optim.Adam(D.parameters(), 0.001)

for epoch in range(3):
    for real, _ in loader:
        real = real.view(-1, 784)

        # labels
        ones = torch.ones(real.size(0), 1)
        zeros = torch.zeros(real.size(0), 1)

        # Train D
        z = torch.randn(real.size(0), 100)
        fake = G(z)

        d_loss = loss(D(real), ones) + loss(D(fake.detach()), zeros)
        d_opt.zero_grad()
        d_loss.backward()
        d_opt.step()

        # Train G
        z = torch.randn(real.size(0), 100)
        fake = G(z)

        g_loss = loss(D(fake), ones)
        g_opt.zero_grad()
        g_loss.backward()
        g_opt.step()

    print("Epoch:", epoch+1)
```

```
Epoch: 1
Epoch: 2
Epoch: 3
```

```python
z = torch.randn(9, 100)
img = G(z).view(-1, 28, 28).detach()

for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(img[i], cmap='gray')
    plt.axis('off')
plt.show()
```



Result :

The GAN successfully generates **realistic handwritten digit images** that resemble the MNIST dataset, demonstrating the model's ability to learn and replicate the underlying data distribution.